

AI Service Desk

Technical Documentation by Gokul M

(Trainee - MTS Full Stack)

Date: July 24, 2026 **Version:** 1.0

Version	Date	Author	Description
1.0.0	July 24, 2026	Gokul M	Python Backend Training task

Table of Contents

Technical Documentation	
1. Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Intended Audience	1
1.4 References	1
1.5 Acronyms and Definitions	1
2. System Overview	2
2.1 High-Level Architecture	2
2.2 System flow - Key Components	2
2.3 User Roles and Access Levels	2
3. Functional Requirements	2
3.1 Module/Feature Name	2
3.2 Inputs/Outputs	3
3.3 Business Rules	3
3.4 Error Handling	3
4. Non-Functional Requirements	3
4.1 Performance	3
4.2 Security	3
4.3 Availability	4
4.4 Maintainability	4
5. Data Model / Database Design	4
5.1 Entity-Relationship Diagram	5
5.2 Tables and Fields	5
5.3 Indexes and Keys	5
5.4 Sample Data	5
6. Integration Points	5
6.1 External Systems	5
6.2 APIs Used	5
6.3 Data Flow Diagram	5
6.4 Authentication/Authorization	6

7. Test Strategy	6
7.1 Test Scenarios	6
7.2 Expected Results	6
7.3 Traceability to Requirements	6
8. Deployment & Configuration	6
8.1 Environments	6
8.2 Setup Steps	7
8.3 Configuration Files and Parameters	7
9. Appendix	8
Glossary	8
Code Snippets	9
Screenshots	10
References	13

1. Introduction

1.1 Purpose

This document describes the technical implementation of the AI Service Desk API, a FastAPI-based backend service for managing support tickets with AI-powered summarization and response suggestions.

1.2 Scope

The system provides RESTful APIs for ticket management (create, read, update, delete) and AI-powered ticket analysis using Amazon Bedrock. It includes database persistence, health checks, error handling, and comprehensive test coverage.

1.3 Intended Audience

- Developers maintaining or extending the system
- DevOps engineers deploying and monitoring the service
- QA engineers verifying functionality
- Product owners understanding technical capabilities

1.4 References

- FastAPI Documentation: <https://www.tutorialspoint.com/fastapi>
- SQLAlchemy Documentation: <https://docs.sqlalchemy.org/>
- AWS Bedrock Documentation: <https://docs.aws.amazon.com/bedrock/>
- Pydantic Documentation: <https://docs.pydantic.dev/>

1.5 Acronyms and Definitions

Acronym	Definition
API	Application Programming Interface
ORM	Object-Relational Mapping
DTO	Data Transfer Object
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
AWS	Amazon Web Services
Bedrock	AWS service for generative AI

2. System Overview

2.1 High-Level Architecture

The system follows a layered architecture:

- API Layer: FastAPI routers handling HTTP requests/responses
- Service Layer: Business logic for ticket operations and AI services
- Repository Layer: Data access abstraction over SQLAlchemy ORM (PostgreSQL/SQLAlchemy interactions)
- External Services integration (Bedrock for AI)
- Cross-cutting: Exception handling, middleware, configuration

2.2 System Flow - Key Components

- **API Entry Point:** FastAPI application with versioned API routes (v1,...)(/tickets)
- **Request Validation:** Pydantic models validate incoming request data
- **Service Layer:** Business logic orchestrates ticket creation/retrieval/update/deletion, status transition validation, and AI service integration for summarization
- **Data Access:** Repository pattern with SQLAlchemy ORM
- **External Integration:** AWS Bedrock service for AI capabilities
- **Response Formation:** Pydantic models format outgoing responses
- **Cross-cutting Concerns:** Exception handling converts domain exceptions to HTTP responses; middleware adds response timing and profiling; created health/readiness endpoints for monitoring

2.3 User Roles and Access Levels

The system currently implements role-less access control where all authenticated users have full access to all endpoints. Authentication is not implemented in the current version, focusing on core functionality.

3. Functional Requirements

3.1 Module: Ticket Management

Purpose: Provides CRUD operations for support tickets with status tracking and assignment capabilities.

Functionality:

- Create new tickets with title, priority, and contact email
- Retrieve tickets with optional filtering by status and priority
- Update ticket fields including status transitions with validation
- Delete tickets
- Automatic timestamp tracking for creation

3.2 Inputs/Outputs

Create Ticket

- Input: title (string, 3-100 chars), priority (low/medium/high), email (valid format)
- Output: Ticket object with ID, timestamps, and default status "open"

Get Tickets

- Input: Optional status and priority query parameters
- Output: List of ticket objects matching filters

Update Ticket

- Input: Ticket ID and optional fields to update (title, priority, status, assignee, email)
- Output: Updated ticket object
- Business Rule: Cannot transition from "resolved" to another status

Delete Ticket

- Input: Ticket ID
- Output: Success confirmation message

3.3 Business Rules

- Ticket titles must be 3-100 characters after trimming whitespace
- Priority must be one of: low, medium, high
- Status must be one of: open, in_progress, resolved
- Email must be valid format when provided
- Cannot transition a resolved ticket to open or in_progress status
- New tickets always start with "open" status
- Timestamps are automatically set on creation

3.4 Error Handling

- Custom exception hierarchy extends ServiceDeskException
- TicketNotFoundException: Returns 404 when ticket ID doesn't exist
- InvalidTicketStatusTransitionException: Returns 400 for invalid status transitions
- Validation errors from Pydantic models return 422 with details
- Global exception handler formats errors as JSON with status code and message
- AI service failures return 502 Bad Gateway with generic message

4. Non-Functional Requirements

4.1 Performance

- Health check endpoint responds in <100ms under normal conditions
- Database operations use async SQLAlchemy for non-blocking I/O
- API endpoints designed for concurrent request handling via FastAPI/Starlette
- Response time middleware tracks and reports request processing time

4.2 Security

- CORS middleware configured for controlled cross-origin access
- Input validation prevents injection and malformed data
- Error messages avoid exposing internal implementation details
- Secrets managed via environment variables (AWS credentials, database URLs)

- Passwords/secrets not logged or exposed in responses
- HTTPS recommended for production deployment (handled at reverse proxy level)

4.3 Availability

- Health check endpoint (**/health**) verifies database connectivity
- Readiness endpoint (**/ready**) indicates service readiness
- Automatic fallback to in-memory SQLite database when PostgreSQL unavailable
- Graceful degradation: AI service failures don't break ticket operations
- Stateless horizontal scaling possible (except for database affinity)

4.4 Maintainability

- Separation of concerns via layered architecture
- Clear separation: API contracts, business logic, data access
- Configuration centralized in Pydantic settings model
- Database migrations managed via Alembic
- Comprehensive test coverage with unit and integration tests
- Dependency injection for testability (Bedrock service client injection)
- Type hints throughout for IDE support and error prevention

5. Data Model / Database Design

5.1 Entity-Relationship Diagram

Entity: Ticket

Field	Type
id	Integer (PK)
title	String(100)
priority	Enum [low, medium, high]
status	Enum [open, in_progress, resolved] (default: open)
assignee	String(255, nullable)
created_at	DateTime (default: utcnow)
email	String(255, nullable)

5.2 Tables and Fields

Table: tickets

- id: Primary key, auto-incrementing integer
- title: Non-nullable string (max 100 chars)

- priority: Non-nullable enum (low/medium/high)
- status: Non-nullable enum (open/in_progress/resolved, default: open)
- assignee: Nullable string (max 255 chars)
- created_at: Non-nullable datetime (default: utcnow)
- email: Nullable string (max 255 chars)

5.3 Indexes and Keys

- Primary key index on id column
- Index on id (automatic from primary key)
- No additional indexes currently defined (may be added based on query patterns)

5.4 Sample Data

id	title	priority	status	assignee	created_at	email
1	Database connection failing	high	open	null	2026-07-24T10:30:00Z	admin@example.com
2	Password reset request	medium	in_progress	support_team	2026-07-24T09:15:00Z	user@example.com

6. Integration Points

6.1 External Systems

- Amazon Bedrock: AI service for ticket summarization and response generation
- PostgreSQL: Primary relational database (with SQLite fallback for development/testing)

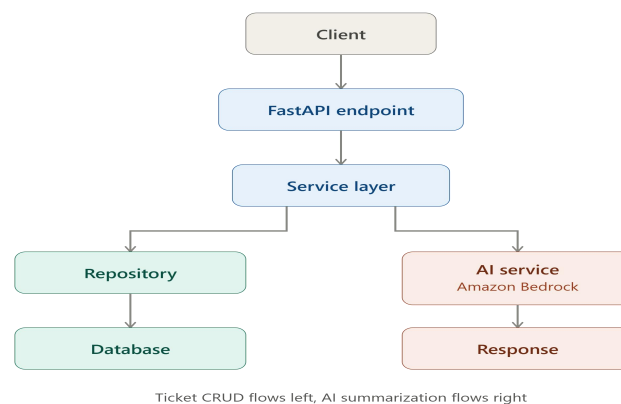
6.2 APIs Used

- **Internal:** Self-contained REST API with versioned endpoints (/v1/tickets, /v1/ai)
- **External:** AWS Bedrock Converse API for AI text generation; no external APIs consumed beyond AWS and database drivers

Method	Endpoint	Description
POST	/tickets/	Create a new ticket (title, priority, email)
GET	/tickets/	List tickets, optional ?status= and ?priority= filters
GET	/tickets/{id}	Retrieve a single ticket by ID

PUT / PATCH	/tickets/{id}	Update ticket fields (title, priority, status, assignee, email)
DELETE	/tickets/{id}	Delete a ticket by ID
POST	/ai/summarize	AI-powered ticket summarization via BedrockService → Bedrock Converse API
GET	/health	Health check — verifies database connectivity
GET	/ready	Readiness check — indicates service is ready to accept traffic
GET	/docs	Auto-generated Swagger/OpenAPI documentation UI

6.3 Data Flow Diagram



6.4 Authentication/Authorization

Currently not implemented in our system .

7. Test Strategy

7.1 Test Scenarios

Unit Tests

- Pydantic model validation (email, title length, enum values)
- Exception formatting and message correctness
- Property-based testing for computed fields

Integration Tests

- Ticket creation with valid/invalid data
- Title whitespace trimming validation
- Status filtering and retrieval
- Update operations including status transitions
- Delete operations
- Error conditions (not found, invalid transitions)

7.2 Expected Results

- All validation rules enforced correctly
- Proper HTTP status codes returned (200, 201, 400, 404, 422, 502)
- Response bodies match defined Pydantic models
- Database state changes correctly reflect operations
- Error messages are clear and actionable
- AI service failures handled gracefully without crashing ticket operations

7.3 Traceability to Requirements

- Test cases map directly to functional requirements in Section 3
- Validation tests cover business rules (Section 3.3)
- Error handling tests validate exception responses (Section 3.4)
- Integration tests verify end-to-end API functionality
- Performance characteristics validated through test execution timing

8. Deployment & Configuration

8.1 Environments

- Development: postgresQL db through Docker Image
- Testing: In-memory SQLite for isolated tests
- Staging/Production: PostgreSQL with connection pooling
- All environments configurable via environment variables

8.2 Setup Steps

- Clone repository

```
git clone https://github.com/Gokul-272/python-backend-Ai-Ticket-Service
```

Create a .env file with required variables

Install dependencies:

```
pip install -r requirements.txt
```

Run database migrations:

```
alembic upgrade head
```

Start the application:

```
uvicorn app.main:app --reload
```

Access the API at <http://localhost:8000>

View documentation at <http://localhost:8000/docs>

8.3 Configuration Files and Parameters

Required Environment Variables:

Variable	Description
APP_NAME	Application name (default: "AI Service Desk")
API_VERSION	API version (default: "1.0.0")
DEBUG	Boolean flag for debug mode
DATABASE_URL	PostgreSQL connection string
AWS_ACCESS_KEY_ID	AWS credentials for Bedrock
AWS_SECRET_ACCESS_KEY	AWS credentials for Bedrock
AWS_REGION_NAME	AWS region (e.g., "us-east-1")
BEDROCK_MODEL_ID	Bedrock model identifier

Configuration File: `pyproject.toml` contains project dependencies and metadata.

Configuration Class: `app.core.config.Settings` uses Pydantic for type-safe environment variable parsing with `.env` file support.

9. Appendix

Glossary

Term	Definition
DTO (Data Transfer Object)	Object used to encapsulate data for transfer between system layers
ORM (Object-Relational Mapping)	Technique for converting data between relational databases and object-oriented models
REST (Representational State Transfer)	Architectural style for networked applications
JWT (JSON Web Token)	Open standard for securely transmitting information as a JSON object

Code Snippets

Ticket Creation Endpoint

```
@app.post("/tickets/", response_model=TicketResponse, status_code=201)
async def raise_ticket(ticket: CreateTicketRequest, db: AsyncSession = Depends(get_db)):
    return await create_ticket(db, ticket)
```

Ticket Model Validation

```
class Ticket(Base):
    __tablename__ = "tickets"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(100), nullable=False)
    priority = Column(SQLEnum(PriorityEnum), nullable=False)
    status = Column(SQLEnum(StatusEnum), nullable=False, default=StatusEnum.OPEN,)
    assignee = Column(String(255), nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow, nullable=False,)
    email = Column(String(255), nullable=True)

    @validates("title")
    def validate_title(self, key, value):
        if not isinstance(value, str):
            raise TypeError("Title must be a string")
        return value
```

Exception Handler

```
@app.exception_handler(ServiceDeskException)
async def service_desk_exception_handler(request, exc: ServiceDeskException):
    return JSONResponse(status_code=exc.status_code, content={"detail": exc.message})
```

Database Connection with Fallback

```
@asynccontextmanager
async def lifespan(app):
    try:
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)
    except Exception:
        use_in_memory_fallback()
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)
    Yield
```

Screenshots

1. Health check

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8080/health' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8080/health
```

Server response

Code	Details
200	Response body <pre>{ "status": "healthy", "service": "AI Service Desk", "version": "1.0.0", "timestamp": "2026-07-24T05:57:58.311630+00:00" }</pre> Response headers <pre>application: AI Service Desk content-length: 113 content-type: application/json date: Fri, 24 Jul 2026 05:57:57 GMT response-time-taken: 5.26 ms server: uvicorn</pre>

2. Readiness Check

Request URL

```
http://127.0.0.1:8080/ready
```

Server response

Code	Details
200	Response body <pre>{ "status": " Ready ,All Set to Goooo!" }</pre> Response headers <pre>application: AI Service Desk content-length: 38 content-type: application/json date: Fri, 24 Jul 2026 06:01:03 GMT response-time-taken: 0.88 ms server: uvicorn</pre>

3. Create Ticket

Request URL

```
http://127.0.0.1:8000/tickets/
```

Server response

Code	Details
201	Response body <pre>{ "id": 1, "title": "DOCS_TESTING", "priority": "high", "status": "open", "created_at": "2026-07-24T06:02:15.895066", "assignee": null, "email": "Gokull1@gmail.com", "is_resolved": false }</pre> Response headers <pre>access-control-allow-credentials: true application: AI Service Desk content-length: 171 content-type: application/json date: Fri, 24 Jul 2026 06:02:15 GMT response-time-taken: 32.24 ms server: uvicorn</pre>

4. Get Ticket

Request URL

```
http://127.0.0.1:8000/tickets/view
```

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "title": "DOCS_TESTING", "priority": "high", "status": "open", "created_at": "2026-07-24T06:02:15.895066", "assignee": null, "email": "Gokull1@gmail.com", "is_resolved": false }]</pre> Response headers <pre>application: AI Service Desk content-length: 173 content-type: application/json date: Fri, 24 Jul 2026 06:07:10 GMT response-time-taken: 5.68 ms server: uvicorn</pre>

5. Update Ticket

Request URL

```
http://127.0.0.1:8000/tickets/update/1
```

Server response

Code	Details
200	Response body <pre>{ "id": 1, "title": "Update operation", "priority": "low", "status": "open", "created_at": "2026-07-24T06:02:15.895066", "assignee": "gokul", "email": "abcd@gmail.com", "is_resolved": false }</pre> Response headers <pre>access-control-allow-credentials: true application: AI Service Desk content-length: 174 content-type: application/json date: Fri, 24 Jul 2026 06:11:25 GMT response-time-taken: 12.54 ms server: uvicorn</pre>

6. Delete Ticket

Request URL

```
http://127.0.0.1:8000/tickets/delete/1
```

Server response

Code	Details
200	Response body <pre>{ "message": "Ticket deleted successfully" }</pre> Response headers <pre>access-control-allow-credentials: true application: AI Service Desk content-length: 41 content-type: application/json date: Fri, 24 Jul 2026 06:13:14 GMT response-time-taken: 7.21 ms server: uvicorn</pre>

References

Frameworks & Libraries

- FastAPI — <https://fastapi.tiangolo.com/>
- Pydantic — <https://docs.pydantic.dev/>
- SQLAlchemy — <https://docs.sqlalchemy.org/>
- Alembic (database migrations) — <https://alembic.sqlalchemy.org/>
- Uvicorn (ASGI server) — <https://www.uvicorn.org/>
- Starlette (underlying FastAPI) — <https://www.starlette.io/>

Database

- PostgreSQL — <https://www.postgresql.org/docs/>
- SQLite (dev/test fallback) — <https://www.sqlite.org/docs.html>

AI / Cloud Services

- Amazon Bedrock — <https://docs.aws.amazon.com/bedrock/>
- Bedrock Converse API — <https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html>
- AWS SDK for Python (Boto3) — <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>

Testing & Quality

- pytest — <https://docs.pytest.org/>
- pytest-asyncio (for async test support) — <https://pytest-asyncio.readthedocs.io/>
- Locust (load testing) — <https://docs.locust.io/en/stable/>

Standards Referenced

- REST architectural style (Fielding's dissertation) — https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm